The Curse of Sisyphus

Sisyphus was condemned by the gods to roll a boulder up a hill for eternity, only to watch it roll back down each time he neared the top. Not punishment for failure, but punishment for believing he could outsmart the system.

If implementation is the cheap part now, where does the hard part go?

I was the problem.

That took me a while to see. I was shipping fast, quality was good, tests were there. Every review request I opened was a genuine improvement to the codebase. And I had multiple open at any given time, because I was filling the review wait with more implementation. While someone in Canada was sleeping, I was already three tasks deep: a new review request open, another under review, a third being planned.

The team was distributed, six hours between Europe and Canada, but that wasn't the issue. Distributed teams work. The issue was volume. AI let me produce faster than the team could absorb. Review requests piled up. I started stacking them to manage the backlog, which created more noise, not less. The system was designed to produce exactly that outcome, not a slow team, but a process that couldn't keep pace with the output.

I felt the burden. No one said anything directly, but you feel it in the silence. Review requests sitting. Comments sparse. The sense of generating more than the team can hold.

So I did what felt productive. I opened another review request.

The review bottleneck was the same problem wearing different clothes: the knowledge gap.

We had something close to the right process. A product manager and the more senior engineers would talk through what needed building. Decisions got made and intent got written into Confluence. Then it became tickets, and the team picked and went.

That sounds reasonable, but in practice, two things were broken simultaneously.

The scoping documents were written from a product manager's perspective: where we want to go, what the final state should look like, the vision six months out. Useful for a roadmap. Not useful when an engineer is trying to decide how a function should behave today. And when the docs did go technical, they'd sometimes arrive with a solution already decided by the seniors and the product manager, handed down to whoever picked the ticket. Or they'd leave the technical decisions entirely to the engineer, mid-implementation, without the context the seniors had built up in their informal conversations.

We had also stopped doing planning sessions. I'm not sure exactly when. It wasn't a decision. It just faded. The backlog existed, the docs existed, the tickets existed. Planning felt redundant when everything was already written down somewhere.

The missing thing wasn't documentation. It was the conversation that produced it. Better documents preserve understanding; they don't create it. The creating happened in the room. When the room went away, the understanding went with it. The documents stayed. But documents can't answer questions mid-implementation. Documents can't notice when a technical decision made six weeks ago no longer fits what the system actually looks like.

So the scoping document was always incomplete in a specific way. Not wrong at the top; the vision was usually fine. Missing in the middle, where the engineers who hadn't been in the discussion between seniors and the product manager needed to make real decisions, and had nothing to go on except a ticket and a Confluence page written for a different audience.

And then I came along with an AI power tool and started shipping against that gap at three times the previous pace.

The review bottleneck wasn't really about reviews. It was about shared understanding. Nobody could review fast because nobody had been in the room when the thinking happened, and half the thinking had never been written down at all.

Working on JoustMania, the same Bluetooth party game behind the KubeCon talk, we wanted to add a feature that remembered how players had played before. Dynamic behavior that adapted over time. Interesting idea. We implemented it with AI assistance: storage, pipelines, game recording, replay. Each piece followed logically from the previous one. The agent was never wrong about any individual step.

The review request sat open for weeks. We never merged it.

The feature was pointless. JoustMania is a local party game. Controllers get passed around,

sessions end, nobody registers. There's no way to know if the same person is holding the same controller from one game to the next. Player history doesn't attach to anything. We'd built a sophisticated solution to a problem that didn't exist.

Nobody asked that question before the first line of code. We asked it, implicitly, by never merging the review request.

The boulder made it all the way up. Then it just stayed there.

Addy Osmani, writing about loop engineering in June 2026, quoted Boris Cherny, head of Claude Code at Anthropic: "I don't prompt Claude anymore. I have loops running that prompt Claude and figuring out what to do. My job is to write loops." Osmani's observation: "Two people can build the exact same loop and get completely opposite results. One uses it to move faster on work they understand deeply. The other uses it to avoid understanding the work at all. The loop doesn't know the difference. You do."[1]

The loop amplifies whatever understanding the team brings to it. If the team didn't build shared understanding before the loop ran, the loop executes the misalignment faster. The boulder doesn't just go up faster; it goes up faster in the wrong direction.

Eliyahu Goldratt's Theory of Constraints[2] says one simple thing: optimizing any part of a system that isn't the bottleneck doesn't improve throughput. It just moves work faster toward the constraint. If shared understanding is the bottleneck, if the team's ability to agree on what to build is what limits delivery, then making implementation faster doesn't help. It produces more output against a constraint that hasn't moved.

The boulder goes up faster. The constraint was never the pushing.

Some teams will read this and disagree. They removed the planning sessions, went AI-first, doubled their output, and things are working fine. They might be right. For now. What "for now" means varies: sometimes it's the first time a senior engineer leaves and the system becomes inexplicable to the people who remain. Sometimes it's the first time a feature request surfaces a decision nobody remembered making. Sometimes it's the moment the codebase gets complex enough that the informal knowledge can't hold it together anymore. Speed without shared understanding creates debt that compounds invisibly.

Most teams I see have done the same thing I did. They've added AI to the implementation end of the process and called it transformation.

---

[1] Addy Osmani, "Loop Engineering" (addyosmani.com/blog/loop-engineering/), quoting Boris Cherny of Anthropic.

[2] Eliyahu M. Goldratt, *The Goal* (North River Press, 1984); the Theory of Constraints.

It's just acceleration, and acceleration without a better process gets to the wrong place faster.

The instinct is understandable. AI tools drop into an existing workflow cleanly. The engineer opens an editor, starts prompting, ships more code. The feedback loop is immediate and satisfying. The problems it creates are delayed and diffuse: a review queue that grows quietly, a shared understanding that gets thinner as the pace increases, a team that starts to feel like an audience for one person's output.

Changing the process is a different kind of work. It requires the team to agree that something is broken before the pain is obvious. It requires investment in planning before anyone has written a line of code. It requires trusting that the time spent in a room together, before the agent runs, is the most valuable engineering time a team has, and that's a hard sell when the backlog is full and the AI tool is right there.

The agent loop is real. AI can take a well-specified ticket, implement it, run a review pass, flag what doesn't fit, and cycle back, without a human in the middle. That's not science fiction. It's available now, and it works when the input is good enough.

That loop is the agent's: implementation, review, iteration, at machine speed. To its right sits production, where the output meets reality. To its left is where the thinking happens: what to build, why, and what done looks like. This book is an argument for the left of the loop. For the humans who need to be there while the work is still on the page, building the shared understanding that determines whether the loop produces the right thing.

The input is the problem.

OpenAI described this precisely after shipping roughly one million lines of production code with Codex agents: "from the agent's point of view, anything it cannot access in-context effectively does not exist."[3] Knowledge in Slack threads, in documents, in someone's head: invisible to the system. The agent builds from what it can see. What it can't see might as well not exist.

Garbage in, garbage out, just faster. If the prompt reflects a thin scoping document that a product manager wrote on a six-month horizon while everyone else was working on tickets, the output will be confidently, efficiently wrong.

The shift has to happen before a line is written. Collaboration moves left: into planning, into the conversation, into the moment where the team builds shared understanding of what they're actually trying to build.

If the team can be in the same room, or on the same call, that's the strongest version:

[3]OpenAI, "Harness Engineering: Leveraging Codex in an Agent-First World" (openai.com/index/harness-engineering/).

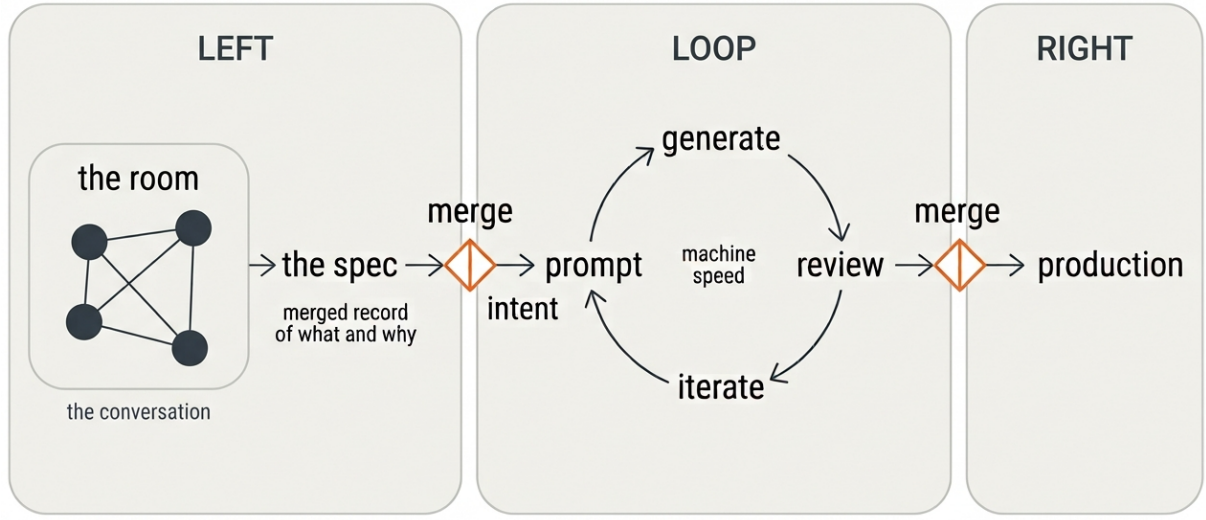

Figure 1: The room merges into a spec; the spec merges into the loop, where the agent generates, reviews, and iterates at machine speed; the loop merges into production.

synchronous, whole team, shared understanding built in real time. Everyone leaves with a spec and everyone in the room owns it.

Not every team has that option. When the timezone gap is six hours, synchronous starts to mean someone sacrificing their morning or their evening on a regular basis. For teams in that situation, the planning needs a different form factor. But the outcome has to be the same. A shared decision, made by everyone who needs to act on it, recorded in a way the agent can work from.

Think of it like a review request. Not on code, on the spec itself. The spec opens for review. Anyone can comment. Decisions get made in the thread, not in a side conversation between the product manager and two seniors. And there's a merge condition: a moment where the team agrees this is decided, this is what we build from, the agent can run.

That last part is what our Confluence documents were missing. They existed. They sat there. But there was no merge gate. No moment where the team said this is decided. No record of why the decision was made, only what it was.

That matters more than it sounds. Six months later, when the system has changed and the decision no longer fits, the team needs to know whether to revisit it or discard it. The reasoning

is what tells them. A document with no comment history and no approval record gives them the what with none of the why. A spec treated like a review request gives them both.

The agent working from a spec like that is in a fundamentally different position. It has context, not just requirements. It can flag when a new ticket contradicts a previous decision. It can surface the reasoning when something looks wrong.

The spec becomes a living record of intent, not a snapshot of what one person thought the product should be.

I'm not writing this from the outside. I was the power user creating the bottleneck. I was shipping against a ticket I hadn't fully questioned, in a team that hadn't fully owned the thinking behind it, across a timezone gap that made everything slower and more isolated.

The process wasn't broken in an obvious way. It was broken in the quiet way that distributed async teams break: everyone doing their part, nobody in the same moment, understanding siloed in the people who wrote the documents rather than shared across the people doing the work.

What I needed wasn't a better AI tool. I needed the team to converge on intent before I started. In a room if possible. Async if not. Together, deliberately, with a record of what we decided and why.

The process was the problem. But the process isn't the only thing AI changes. It also changes what's left for the people inside it.